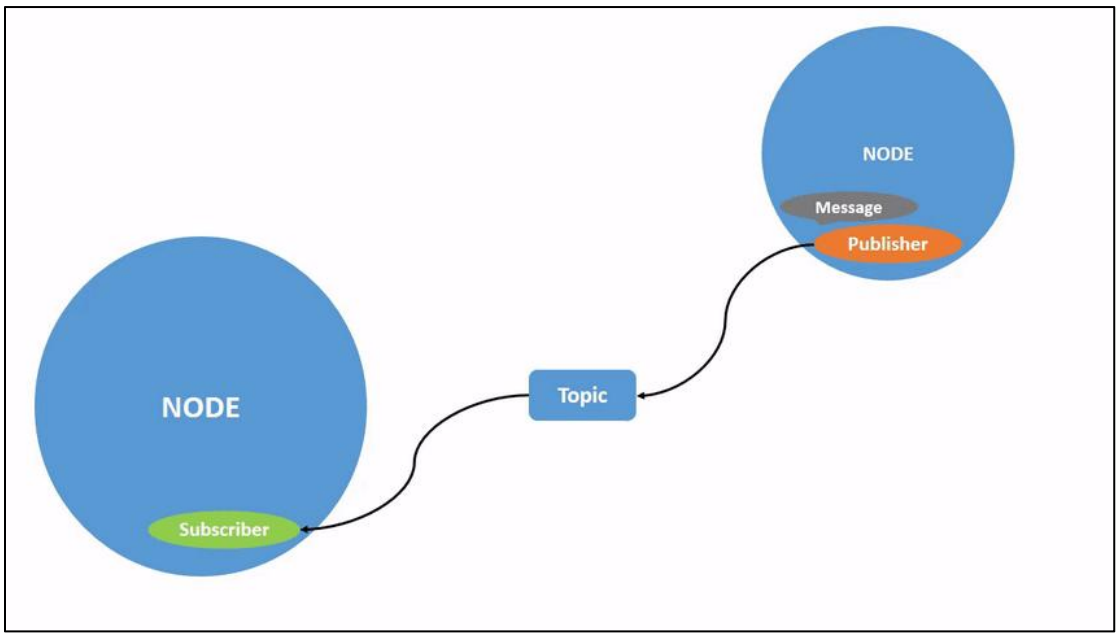
第 8 课 ROS2 话题

1.话题通信简介

话题通信是 ROS2 视频频率最高的一种通信方式，有发布者发布指定话题的数据，订阅者只要订阅了该话题的数据，就可以接收到数据。

话题通信是基于发布/订阅模型，如图：



话题数据传输的特性是从一个节点到另外一个节点，发送数据的对象称之为发布者，接收数据的对象称之为订阅者，每一个话题都需要一个名字，传输的数据也需要有固定的数据类型。

2.创建话题

2.1 创建发布者

1) 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ros2_ws/src/**”指令，切换到 ros2_ws 工作空间的 src 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/
```

2） 输入“**ros2 pkg create topic_demo --build-type ament_python --dependencies rclpy**”并按下回车，创建一个名为“**topic_demo**”的功能包，添加依赖关系 rclpy。

```
ubuntu@ubuntu:~/ros2_ws/src$ ros2 pkg create topic_demo --build-type ament_python --dependencies rclpy
going to create a new package
package name: topic_demo
destination directory: /home/ubuntu/ros2_ws/src
package format: 3
```

3） 输入“**cd topic_demo/topic_demo/**”指令，切换到 topic_demo 功能包下。

```
ubuntu@ubuntu:~/ros2_ws/src$ cd topic_demo/topic_demo/
```

4） 输入指令“**gedit topic_pub.py**”编辑程序，复制下面程序粘贴到文本当汇总，“**ctrl+s**”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/topic_demo/topic_demo$ gedit topic_pub.py
```

```
import rclpy  # 导入 rclpy 模块

from rclpy.node import Node  # 导入 Node 类



from std_msgs.msg import String  # 导入 String 消息类型




class MinimalPublisher(Node):  # 定义一个继承自 Node 类的 MinimalPublisher 类


    def __init__(self):

        super().__init__('minimal_publisher')  # 调用父类构造函数初始化节点

        self.publisher_ = self.create_publisher(String, 'topic', 10)  # 创建一个发布者对象，发布 String 类型的消息到名为'topic'的话题，队列大小为 10

        timer_period = 0.5  # 定时器周期为 0.5 秒
```

```
        self.timer = self.create_timer(timer_period, self.timer_callback)
  # 创建定时器，设置周期为 timer_period，回调函数为 timer_callback
        self.i = 0  # 初始化计数器


    def timer_callback(self):
        msg = String()  # 创建一个 String 类型的消息对象
        msg.data = 'Hello World: %d' % self.i  # 设置消息内容为'Hello Worl
d: i'，其中 i 为计数器的值
        self.publisher_.publish(msg)  # 发布消息到话题
        self.i += 1  # 计数器自增 1



def main(args=None):
    rclpy.init(args=args)  # 初始化 ROS 节点

    minimal_publisher = MinimalPublisher()  # 创建 MinimalPublisher 对象

    rclpy.spin(minimal_publisher)  # 进入主循环

    minimal_publisher.destroy_node()  # 销毁节点
    rclpy.shutdown()  # 关闭 ROS
```

```
if __name__ == '__main__':
    main()
```

5） 输入指令“**chmod +x topic_pub.py**”，并按下回车，为保存的 topic_pub.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/topic_demo/topic_demo$ chmod +x topic_pub.py
```

2.2 创建订阅者

1） 输入指令“**gedit topic_sub.py**”编辑程序，复制下面程序，粘贴到文本当中，然后使用快捷键“ctrl + s ”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/topic_demo/topic_demo$ gedit topic_sub.py
```

```
import rclpy
from rclpy.node import Node


from std_msgs.msg import String


# 定义一个继承自 Node 的 MinimalSubscriber 类
class MinimalSubscriber(Node):


    # 类的初始化方法
    def __init__(self):
        # 调用 Node 类的初始化方法
```

```
        super().__init__('minimal_subscriber')


        # 创建一个订阅者，订阅名为'topic'的 String 类型消息，回调函数为 listene
r_callback，队列大小为 10
        self.subscription = self.create_subscription(String, 'topic', sel
f.listener_callback, 10)


    # 定义消息回调函数
    def listener_callback(self, msg):
        # 获取日志记录器并打印收到的消息
        self.get_logger().info('I heard: "%s"' % msg.data)


# 主函数
def main(args=None):
    # 初始化 ROS 2 节点
    rclpy.init(args=args)


    # 创建 MinimalSubscriber 对象
    minimal_subscriber = MinimalSubscriber()


    # 进入 ROS 2 节点的事件循环
    rclpy.spin(minimal_subscriber)
```

```
    # 销毁节点对象
    minimal_subscriber.destroy_node()


    # 关闭 ROS 2 节点
    rclpy.shutdown()


# 如果该脚本是主程序，则执行 main 函数
if __name__ == '__main__':
    main()
```

2） 输入指令“**chmod +x topic_sub.py**”，并按下回车，为保存的 topic_sub.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/topic_demo/topic_demo$ chmod +x topic_sub.py
```

3.setup.py 文件设置

setup.py 文件是定义一个 ROS2 包的远数据和构建配置，提供了包的元数据、依赖关系、构建配置和安装逻辑等信息，可以帮助开发者正确构建、安装和使用 ROS2 包。需要将 **topic_pub.py** 和 **topic_sub.py** 的程序入口写到 setup.py 文件中。

1） 接着再上一小节的终端当中，输入“**cd ..**”指令，切换到上一级目录。

```
ubuntu@ubuntu:~/ros2_ws/src/topic_demo/topic_demo$ cd ..
ubuntu@ubuntu:~/ros2_ws/src/topic_demo$
```

2） 输入“**gedit setup.py**”指令，并按下回车，打开 setup.py 文件。

```
ubuntu@ubuntu:~/ros2_ws/src/topic_demo$ gedit setup.py
```

3） 输入以下代码到相应的位置上。

```
'topic_custom_pub = topic_demo.topic_custom_pub:main',

'topic_custom_sub = topic_demo.topic_custom_sub:main'
```

```
19      license='TODO: License declaration',
20      tests_require=['pytest'],
21      entry_points={
22          'console_scripts': [
23          'topic_custom_pub = topic_demo.topic_custom_pub:main',
24          'topic_custom_sub = topic_demo.topic_custom_sub:main'
25          ],
26      },
27  )
```

3.编译运行

1） 来到远程桌面，然后新建一个命令行终端，输入“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

```
ubuntu@ubuntu:~/ros2_ws$ colcon build
Starting >>> hello_world_demo
Starting >>> topic_demo
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

4） 输入指令“**ros2 run topic_demo topic_custom_pub**”，并按下回车，启动 topic_pub 话题发布节点。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 run topic_demo topic_custom_pub
```

5） 然后在新建一个新的终端，输入指令

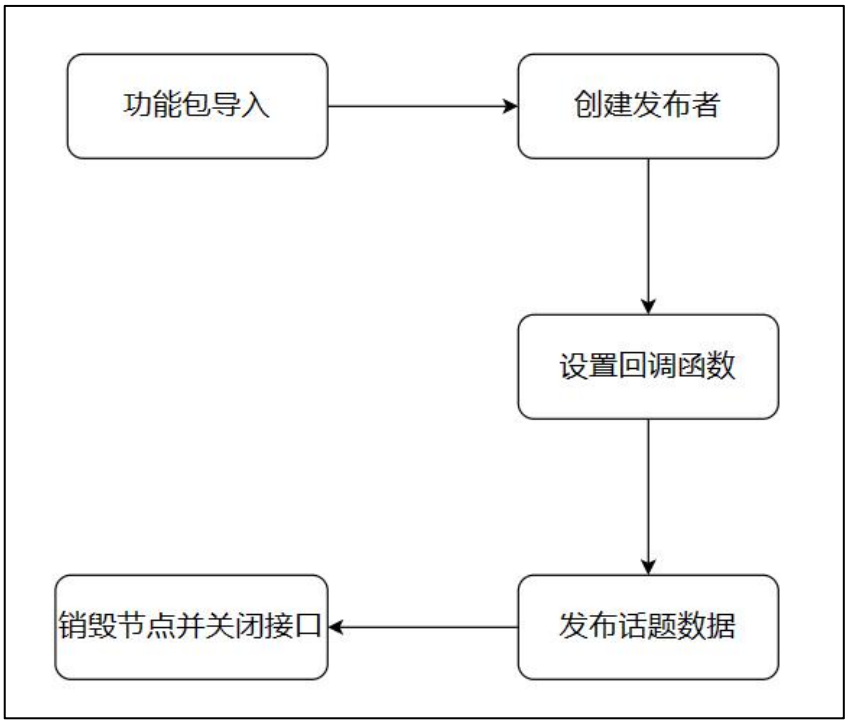
“**ros2 run topic_demo topic_custom_sub**”，并按下回车，启动 topic_sub 话题发布节点。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 run topic_demo topic_custom_sub
[INFO] [1709026628.798449585] [minimal_subscriber]: I heard: "Hello World: 2"
[INFO] [1709026629.281110878] [minimal_subscriber]: I heard: "Hello World: 3"
[INFO] [1709026629.781165951] [minimal_subscriber]: I heard: "Hello World: 4"
[INFO] [1709026630.280742360] [minimal_subscriber]: I heard: "Hello World: 5"
[INFO] [1709026630.780280400] [minimal_subscriber]: I heard: "Hello World: 6"
```

5.程序分析

5.1 话题发布

根据实现的效果，梳理程序的过程逻辑，如下图所示：



创建了一个名为 minimal_publisher 的发布者，该发布者每 0.5 秒发布一个带有递增数字的消息到 topic 话题。在程序运行时，首先会初始化 ROS2 节点，然后创建 MinimalPublisher 对象并进入 ROS 2 节点的事件循环。当程序被中断时，会销毁节点对象并关闭 ROS 2 节点。

◆ 主函数

```python
# 主函数
def main(args=None):
    # 初始化ROS 2节点
    rclpy.init(args=args)

    # 创建MinimalPublisher对象
    minimal_publisher = MinimalPublisher()

    # 进入ROS 2节点的事件循环
    rclpy.spin(minimal_publisher)

    # 销毁节点对象
    minimal_publisher.destroy_node()

    # 关闭ROS 2节点
    rclpy.shutdown()
```

首先调用 rclpy.init()对 ROS2 Python 接口进行初始化，然后实例化 MinimalPublisher()，然后在 ROS2 节点的事件循环执行 minimal_publisher。

◆ MinimalPublisher 类

```python
# 定义一个继承自Node的MinimalPublisher类
class MinimalPublisher(Node):

    # 类的初始化方法
    def __init__(self):
        # 调用Node类的初始化方法，设置节点名称为'minimal_publisher'
        super().__init__('minimal_publisher')

        # 创建一个发布者，发布String类型消息到'topic'，队列大小为10
        self.publisher_ = self.create_publisher(String, 'topic', 10)

        # 创建一个定时器，每0.5秒触发一次timer_callback方法
        timer_period = 0.5  # 秒
        self.timer = self.create_timer(timer_period, self.timer_callback)

        # 计数器，用于生成消息中的数字
        self.i = 0

    # 定义定时器回调函数
    def timer_callback(self):
        # 创建一个String类型的消息
        msg = String()

        # 设置消息数据为'Hello World: 数字'
        msg.data = 'Hello World: %d' % self.i

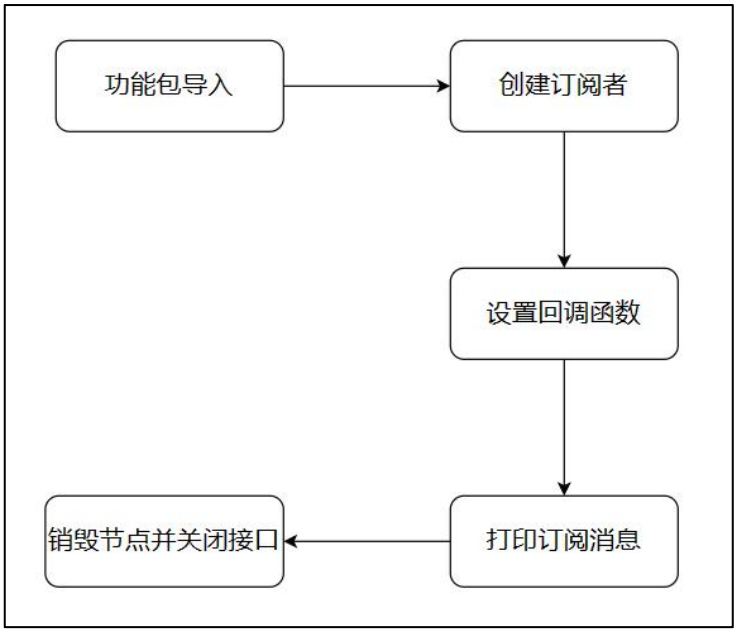
        # 发布消息
        self.publisher_.publish(msg)

        # 计数器自增
        self.i += 1
                                                    10,1          14%
```

首先创建了一个名为 minimal_publisher 的节点，然后创建一个名为 publisher_的发布者，timier_callback()回调函数里面以 0.5 秒的间隔打印带有递增数字的消息到节点的日志中。

5.2 话题订阅

根据实现的效果，梳理程序的过程逻辑，如下图所示：



创建了一个名为 minimal_sublisher 的订阅者，该订阅者打印订阅的消息到日志记录器。在程序运行时，首先会初始化 ROS2 节点，然后创建 MinimalSublisher 对象并进入 ROS 2 节点的事件循环。当程序被中断时，会销毁节点对象并关闭 ROS 2 节点。

◆ 主函数

```
# 主函数
def main(args=None):
    # 初始化ROS 2节点
    rclpy.init(args=args)

    # 创建MinimalSubscriber对象
    minimal_subscriber = MinimalSubscriber()

    # 进入ROS 2节点的事件循环
    rclpy.spin(minimal_subscriber)

    # 销毁节点对象
    minimal_subscriber.destroy_node()

    # 关闭ROS 2节点
    rclpy.shutdown()
```

首先调用 rclpy.init()对 ROS2 Python 接口进行初始化，然后实例化 MinimalSublisher()，然后在 ROS2 节点的事件循环执行 minimal_sublisher。

◆ MinimalSublisher 类

```
# 定义一个继承自Node的MinimalSubscriber类
class MinimalSubscriber(Node):

    # 类的初始化方法
    def __init__(self):
        # 调用Node类的初始化方法
        super().__init__('minimal_subscriber')

        # 创建一个订阅者，订阅名为'topic'的String类型消息，回调函数为listener_callback，队列大小为10
        self.subscription = self.create_subscription(String, 'topic', self.listener_callback, 10)

    # 定义消息回调函数
    def listener_callback(self, msg):
        # 获取日志记录器并打印收到的消息
        self.get_logger().info('I heard: "%s"' % msg.data)
```

首先创建了一个名为 minimal_sublisher 的节点，然后创建一个名为 subscription 的订阅者，listener_callback()回调函数里面打印订阅到的消息内容到日志记录器上。

6.自定义接口

topic_pub.py 和 topic_pub.py 都使用了 ROS2 官方的接口。

```
import rclpy
from rclpy.node import Node

from std_msgs.msg import String

# 定义一个继承自Node的MinimalPublisher类
class MinimalPublisher(Node):

    # 类的初始化方法
    def __init__(self):
```

```
import rclpy
from rclpy.node import Node

from std_msgs.msg import String

# 定义一个继承自Node的MinimalSubscriber类
class MinimalSubscriber(Node):

    # 类的初始化方法
    def __init__(self):
        # 调用Node类的初始化方法
        super().__init__('minimal_subscriber')

        # 创建一个订阅者，订阅名为'topic'的String类型消息，回调函数为listener_callback，队列大小为10
```

虽然使用预定义的接口定义是一种很好的做法，但有时可能还需要定义自己的消息和服务。接下来演示如何创建自定义接口定义的方法。

1） 新建一个命令行终端，然后输入“**cd ~/ros2_ws/src/**”指令，切换到 ros2_ws 工作空间的 src 文件夹中。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/src/
ubuntu@ubuntu:~/ros2_ws/src$
```

2） 输入指令

“**ros2 pkg create demo_interfaces --build-type ament_cmake --dependencies rclpy rclcpp**”并按下回车，创建一个名为“**demo_interfaces**”的功能包。

```
ubuntu@ubuntu:~/ros2_ws/src$ ros2 pkg create demo_interfaces --build-type ament_cmake  --dependencies rclcpp
going to create a new package
package name: demo_interfaces
destination directory: /home/ubuntu/ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['ubuntu <ubuntu@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_cmake
dependencies: ['rclcpp']
creating folder ./demo_interfaces
creating ./demo_interfaces/package.xml
creating source and include folder
creating folder ./demo_interfaces/src
creating folder ./demo_interfaces/include/demo_interfaces
creating ./demo_interfaces/CMakeLists.txt
ubuntu@ubuntu:~/ros2_ws/src$
```

3） 输入“**cd demo_interfaces**”指令，进入自定义接口的功能包。

```
ubuntu@ubuntu:~/ros2_ws/src$ cd demo_interfaces
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$
```

4） 输入“**mkdir msg**”指令，创建文件夹。

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$ mkdir msg
```

5） 输入“**cd msg**”指令，进入 msg 文件夹。

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$ cd msg
```

6） 输入指令“**gedit String.msg**”编辑文档，然后输入以下内容

string data，按住“**ctrl + s**”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces/msg$ gedit String.msg
```



7） 先输入“**cd ..**”返回上一级文件夹：

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces/msg$ cd ..
```

8） 再输入“**gedit CMakeLists.txt**”

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$ gedit CMakeLists.txt
```

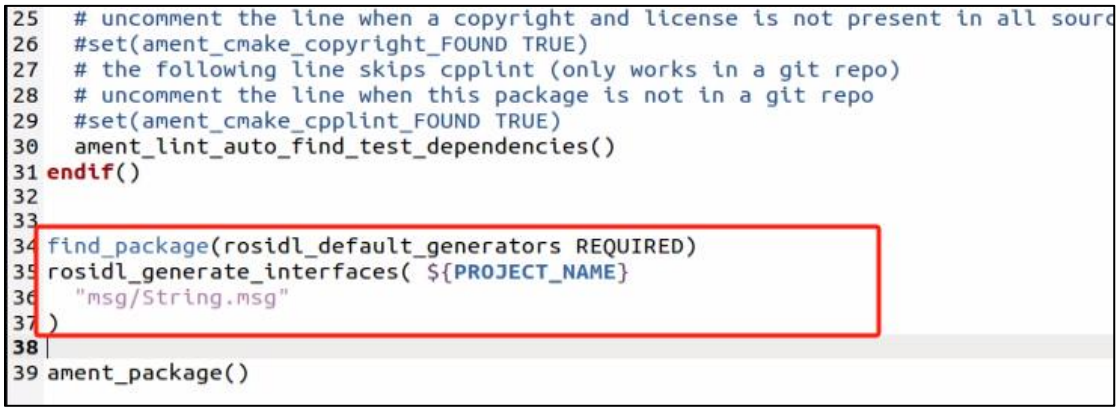
复制下面程序，在下图所示位置添加代码,然后保存并退出即可。

```
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces( ${PROJECT_NAME}

  "msg/String.msg"

)
```

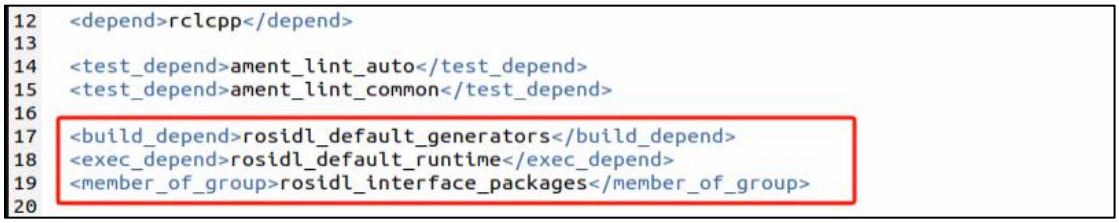


9） 再输入“**gedit package.xml**”，复制下面程序，在下图所示位置添加代码。如需修改，再按下“i”即可修改。修改完成，按下“Esc”，输入“：wq”保存并退出。

```
<build_depend>rosidl_default_generators</build_depend>

<exec_depend>rosidl_default_runtime</exec_depend>

<member_of_group>rosidl_interface_packages</member_of_group>
```



10）参考 3.编译运行的 1）、2）、3）步骤，对工作空间进行编译。

11）将使用 gedit 命令修改 路径为“**/home/ubuntu/ros2_ws/src/topic_demo/topic_demo**”的 **topic_pub.py 内容，**红色方框内的代码修改成“**from demo_interfaces.msg import String**”语句，即可完成自定义消息的使用，实现效果与原来的一致。